

Capítulo 14

Outros Tópicos sobre C

Exercício de Autorrevisão — Resolução Didática

“O Capítulo 14 é o capítulo da coleção de recursos avançados da linguagem C: redirecionamento de entrada/saída, listas de argumentos variáveis, argumentos da linha de comando, compilação de múltiplos arquivos-fonte, término de programas, qualificador *volatile*, sufixos de constantes, arquivos temporários, tratamento de sinais, *calloc/realloc* e o sempre polêmico *goto*. Este exercício testa, de uma só vez, o vocabulário técnico de todo o capítulo.”

1 Exercício 14.1 — Vocabulário do Capítulo 14

Preencha os espaços em cada uma das 19 sentenças (a)–(s).

Antes de iniciarmos item a item, convém organizar o assunto em *quatro blocos temáticos* que correspondem às quatro grandes ideias do capítulo:

Bloco 1: Conexão do programa com o sistema operacional — redirecionamento de E/S, *pipes*, argumentos da linha de comando, sinais e término controlado de programas (itens *a–d*, *i–m*, *p*, *q*).

Bloco 2: Listas de argumentos variáveis — como construir funções que aceitam um número arbitrário de argumentos (itens *e–h*).

Bloco 3: Construção de programas em larga escala — *make* e *makefile* (item *k*).

Bloco 4: Recursos finos de memória, arquivos e tipos — sufixos de constantes, arquivos temporários, *calloc* e *realloc* (itens *n*, *o*, *r*, *s*).

Esses quatro blocos correspondem às Seções 14.2 a 14.12 do livro. Vamos percorrer as 19 sentenças nessa ordem temática.

Bloco 1 — Conexão com o sistema operacional

(a) Símbolo para redirecionar dados de *entrada* a partir de um arquivo

Resposta: redirecionamento de entrada (<).

Em UNIX, Linux e em janelas de comando do Windows, escrever

```
./meuPrograma < dados.txt
```

faz com que o conteúdo de `dados.txt` seja apresentado ao programa *como se tivesse sido digitado no teclado*. O programa em si nada precisa saber sobre essa substituição — continua a ler de `stdin` normalmente. É uma das mais elegantes ideias do projeto UNIX: a **abstração de *stream***, já apresentada no Capítulo 1, permite que a fonte real dos dados seja trocada “por fora” do programa.

Observação de Engenharia de Software

Note a economia conceitual: a função `scanf` que estudamos no Capítulo 2 funciona identicamente em ambos os cenários (teclado ou arquivo). Esse é um exemplo perfeito do princípio de *ocultamento de informações*: o programa não precisa saber *de onde* vêm os dados, apenas *que* chegam por `stdin`.

(b) Símbolo para redirecionar a *saída* da tela para um arquivo

Resposta: redirecionamento de saída (`>`).

Simetricamente ao item anterior, o comando

```
./meuPrograma > resultado.txt
```

faz com que tudo o que o programa imprimiria com `printf` vá parar em `resultado.txt`, em vez de aparecer no terminal. Se o arquivo *já existia*, ele é *truncado* (apagado) antes da escrita — detalhe importante, que motiva o item seguinte.

Erro Comum de Programação

Esquecer que `>` sobrescreve o arquivo é uma das maneiras mais comuns de *perder dados acidentalmente* em uma sessão de terminal. Para preservar o conteúdo anterior, use `»` (item c) ou renomeie o arquivo de destino.

(c) Símbolo para *acrescentar* a saída ao final de um arquivo

Resposta: acréscimo de saída (`»`).

O operador `»` preserva o conteúdo já existente no arquivo e acrescenta a nova saída ao final. É o mecanismo natural para construir *logs*, registros históricos ou arquivos de auditoria. Se o arquivo de destino não existir, ele é criado — então `»` comporta-se exatamente como `>` *na primeira invocação* e como “acrescentar” nas seguintes.

(d) Mecanismo que direciona a saída de um programa para a entrada de outro

Resposta: *pipe* (símbolo `|`).

Os *pipes* são a contribuição mais influente do UNIX para a forma como pensamos a composição de programas. Em vez de monolitizar tarefas dentro de um único programa enorme, fabricamos pequenos utilitários especializados e os encadeamos:

```
./gerador | ./filtro | ./formatador > saida.txt
```

A saída padrão de cada programa torna-se a entrada padrão do programa seguinte. É como ligar tubos hidráulicos — daí o nome.

Observação de Engenharia de Software

A filosofia UNIX de “*escreva programas que façam uma coisa e a façam bem; escreva programas que cooperem*” (Doug McIlroy) ganha vida através dos *pipes*. Boa parte da potência do ambiente de linha de comando vem dessa simples ideia.

Bloco 2 — Listas de argumentos variáveis**(e) Indicador de número variável de argumentos na lista de parâmetros**

Resposta: reticências (...).

A declaração

```
double media(int n, ...);
```

sinaliza ao compilador que, após o parâmetro nomeado `n`, pode haver *zero ou mais* argumentos adicionais, de tipos a princípio desconhecidos do compilador. A própria função *precisa* ter pelo menos um parâmetro nomeado antes das reticências — e é papel desse parâmetro (`n`, no exemplo) informar ao corpo da função *quantos* argumentos adicionais ele deve consumir.

(f) Macro que deve ser chamada antes que os argumentos variáveis possam ser acessados

Resposta: `va_start`.

A macro `va_start`, declarada em `<stdarg.h>`, inicializa uma variável do tipo `va_list`, posicionando-a logo após o último parâmetro nomeado da função. Sintaxe:

```
va_start(args, ultimoParametroNomeado);
```

A partir desse ponto, `args` “aponta” para o primeiro dos argumentos variáveis.

(g) Macro que acessa cada argumento individual da lista variável

Resposta: `va_arg`.

`va_arg(args, tipo)` avalia para o próximo argumento da lista variável, interpretando-o como sendo do tipo especificado, e avança `args` para o próximo. É aqui que reside a *responsabilidade do programador*: o compilador não tem como verificar se o tipo passado a `va_arg` de fato corresponde ao argumento real. Se houver discrepância, o comportamento é indefinido.

Erro Comum de Programação

Solicitar `va_arg(args, int)` quando o argumento real é um `double` é um erro frequente e *cataclísmico* — interpreta-se uma sequência de bits como número de tipo errado, e os valores extraídos passam a ser puro lixo. Documente cuidadosamente, em comentários da função, o protocolo de tipos esperado.

(h) Macro que finaliza o acesso à lista variável

Resposta: `va_end`.

A macro `va_end(args)` faz a “limpeza” interna do `va_list`, permitindo que a função retorne normalmente. Por mais que em muitas implementações `va_end` expanda para nada ou quase nada, o padrão ISO obriga seu uso para garantir portabilidade.

Boa Prática de Programação

O trio `va_start` / `va_arg` / `va_end` aparece sempre na mesma ordem estável: *inicializar*, *percorrer*, *finalizar*. Trate-os como um padrão indissociável — na prática, `va_end` é tão essencial quanto fechar um arquivo com `fclose`.

(i) Argumento de `main` que recebe a *quantidade* de argumentos da linha de comando

Resposta: `argc` (*argument count*).

`argc` é sempre ≥ 1 , porque o próprio *nome de invocação* do programa conta como o primeiro argumento. Se o usuário digitar

```
./meuPrograma arquivo.txt -v
```

então `argc` valerá 3.

(j) Argumento de `main` que armazena os argumentos como strings

Resposta: `argv` (*argument vector*).

`argv` é um array de ponteiros para `char` — ou seja, um array de strings. No exemplo acima, teríamos:

- `argv[0] == "./meuPrograma"`
- `argv[1] == "arquivo.txt"`
- `argv[2] == "-v"`
- `argv[3] == NULL` (terminador convencional)

Observação de Engenharia de Software

A escolha da assinatura `int main(int argc, char *argv[])` é um dos pequenos gênios do projeto de C. Ela permite que *qualquer* programa receba dados externos *antes mesmo de começar a executar*, sem nada além do que o próprio sistema operacional supõe. Voltaremos a essa assinatura em todos os exercícios em que o programa precise de configuração externa.

(l) Função que força o programa a terminar

Resposta: `exit`.

`exit(status)`, declarada em `<stdlib.h>`, encerra a execução do programa *imediatamente* a partir do ponto em que for chamada — mesmo que esteja no meio de uma função aninhada profundamente. Antes de devolver o controle ao sistema operacional, `exit` executa, em ordem:

1. Todas as funções registradas com `atexit` (item *m*, na ordem inversa de registro).
2. O *flush* de todos os *streams* de E/S buferizados.
3. O fechamento de todos os arquivos abertos.
4. A remoção dos arquivos criados por `tmpfile` (item *o*).

O valor `status` é devolvido ao processo invocador (geralmente a *shell*). Por convenção, 0 (ou `EXIT_SUCCESS`) indica sucesso; qualquer valor diferente, fracasso.

(m) Função que registra uma função a ser chamada no término normal

Resposta: `atexit`.

`atexit(func)` adiciona `func` à lista de funções que serão executadas quando o programa terminar normalmente (seja por retorno de `main`, seja por `exit`). Até 32 funções podem ser registradas pelo padrão — mais que suficiente, na prática. Elas são executadas *em ordem inversa de registro*, análoga a uma pilha (LIFO), pois geralmente a última inicialização a ser feita é a primeira a ser desfeita.

Boa Prática de Programação

Use `atexit` para tarefas de *limpeza global*: gravar logs finais, liberar recursos compartilhados, fechar conexões. Mas evite lógica complexa em tratadores `atexit` — eles são executados em um contexto em que muitas estruturas já podem estar parcialmente desmontadas.

(p) Função usada para *interceptar* eventos inesperados

Resposta: `signal`.

A função `signal(sinal, tratador)`, declarada em `<signal.h>`, associa uma função `tratador` a um determinado `sinal`. Quando o sinal ocorrer durante a execução — por causa externa (`<Ctrl>+<C>` do usuário) ou interna (divisão por zero, violação de memória) —, em vez do tratamento padrão do sistema operacional, *nosso* tratador será chamado. É o mecanismo pelo qual programas robustos lidam com situações excepcionais sem ser destruídos por elas.

(q) Função que *gera* um sinal a partir do próprio programa

Resposta: `raise`.

`raise(sinal)` dispara, dentro do programa que a chama, o sinal especificado — como se ele tivesse vindo de fora. É útil para testar tratadores ou para sinalizar, a outras partes do próprio programa, condições excepcionais sintetizadas internamente.

Bloco 3 — Construção de programas em larga escala

(k) Utilitário UNIX/Linux que processa um arquivo de regras de compilação

Resposta: `make`, que lê um arquivo chamado `Makefile` (ou `makefile`).

Quando um programa é composto por muitos arquivos-fonte (`.c` e `.h`), recompilar tudo manualmente a cada alteração é tedioso e ineficiente. O utilitário `make` lê o `Makefile`, que descreve:

- Os *alvos* (*targets*) que se quer construir (executavel final, arquivos-objeto intermediários).
- As *dependências* de cada alvo (quais arquivos precisam estar prontos antes).
- Os *comandos* a executar para construir cada alvo.

`make` então recompila apenas o que mudou e o que depende do que mudou, na ordem correta. É uma ferramenta clássica que se mantém essencial mais de quatro décadas após sua criação.

Bloco 4 — Recursos finos de memória, arquivos e tipos

(n) Indicador de tipo exato anexado a uma constante numérica

Resposta: sufixo.

Em C, a constante literal 42 é, por padrão, um `int`; 3.14 é um `double`. Mas às vezes precisamos forçar um tipo específico:

Sufixo	Tipo forçado
L ou l	<code>long int</code> (em constantes inteiras) ou <code>long double</code> (em constantes reais)
U ou u	<code>unsigned int</code>
UL ou LU	<code>unsigned long int</code>
F ou f	<code>float</code>

Exemplos: 1000000L, 3.14F, 0xFFFFFFFFFU.

(o) Função que cria um arquivo temporário

Resposta: `tmpfile`.

`tmpfile()`, declarada em `<stdio.h>`, abre um arquivo binário para leitura/escrita ("`wb+`") que existe apenas durante a execução do programa. Ele não tem nome visível no sistema de arquivos e é automaticamente removido quando: (i) o arquivo é fechado com `fclose`, ou (ii) o programa termina. É ideal para scratch space temporário, como veremos no Exercício 14.5.

Dica de Portabilidade

Em alguns sistemas, `tmpfile()` pode falhar se o sistema operacional restringir o número de arquivos temporários simultâneos. Sempre verifique o retorno: `NULL` indica falha.

(r) Função que aloca um array zerando os elementos

Resposta: `calloc`.

`calloc(quantidade, tamanho)` aloca espaço para `quantidade` elementos de `tamanho` bytes cada um e *zera* toda a região alocada. Comparada com `malloc(quantidade * tamanho)`, oferece duas vantagens:

1. Inicialização a zero garantida (com `malloc` o conteúdo é lixo).
2. Multiplicação `quantidade * tamanho` feita pela biblioteca, com proteção contra *overflow* em implementações modernas.

(s) Função que muda o tamanho de uma área dinamicamente alocada

Resposta: `realloc`.

`realloc(ptr, novoTamanho)` redimensiona o bloco previamente apontado por `ptr`. O algoritmo é o seguinte:

1. Se houver espaço contíguo suficiente, o bloco é estendido (ou contraído) no lugar.
2. Se não houver, um novo bloco é alocado em outra parte da *heap*, o conteúdo é copiado, e o bloco antigo é liberado.

Em ambos os casos, o retorno é o endereço do bloco resultante. Quando o novo tamanho é *menor*, o conteúdo que cabe é preservado e o restante é descartado — exatamente o que faremos no Exercício 14.7.

Erro Comum de Programação

Não escreva `ptr = realloc(ptr, novoTamanho);` diretamente! Se a realocação falhar, `realloc` retorna `NULL` e o bloco original permanece válido — mas você o terá perdido por sobrescrever `ptr`. Use sempre um ponteiro auxiliar:

```
int *novo = realloc(ptr, novoTamanho);
if (novo != NULL) ptr = novo;
```

Síntese conceitual

O Exercício 14.1 é essencialmente um *check-list* de termos técnicos. Estruturá-lo em quatro blocos, como fizemos, ajuda a memorizá-los não como entradas isoladas de um glossário, mas como *partes coerentes* de quatro grandes histórias:

- Programa ↔ SO via E/S redirecionada, *pipes*, `argc/argv`, `exit/atexit`, `signal/raise`.
- Listas variáveis com `...`, `va_list`, `va_start`, `va_arg`, `va_end`.
- Construção em larga escala com `make` e `Makefile`.
- Recursos finos: sufixos, `tmpfile`, `calloc`, `realloc`.

No documento de *Exercícios* colocaremos esses conceitos *em ação*: oito programas que exercitam, em diferentes combinações, todos os recursos catalogados aqui.